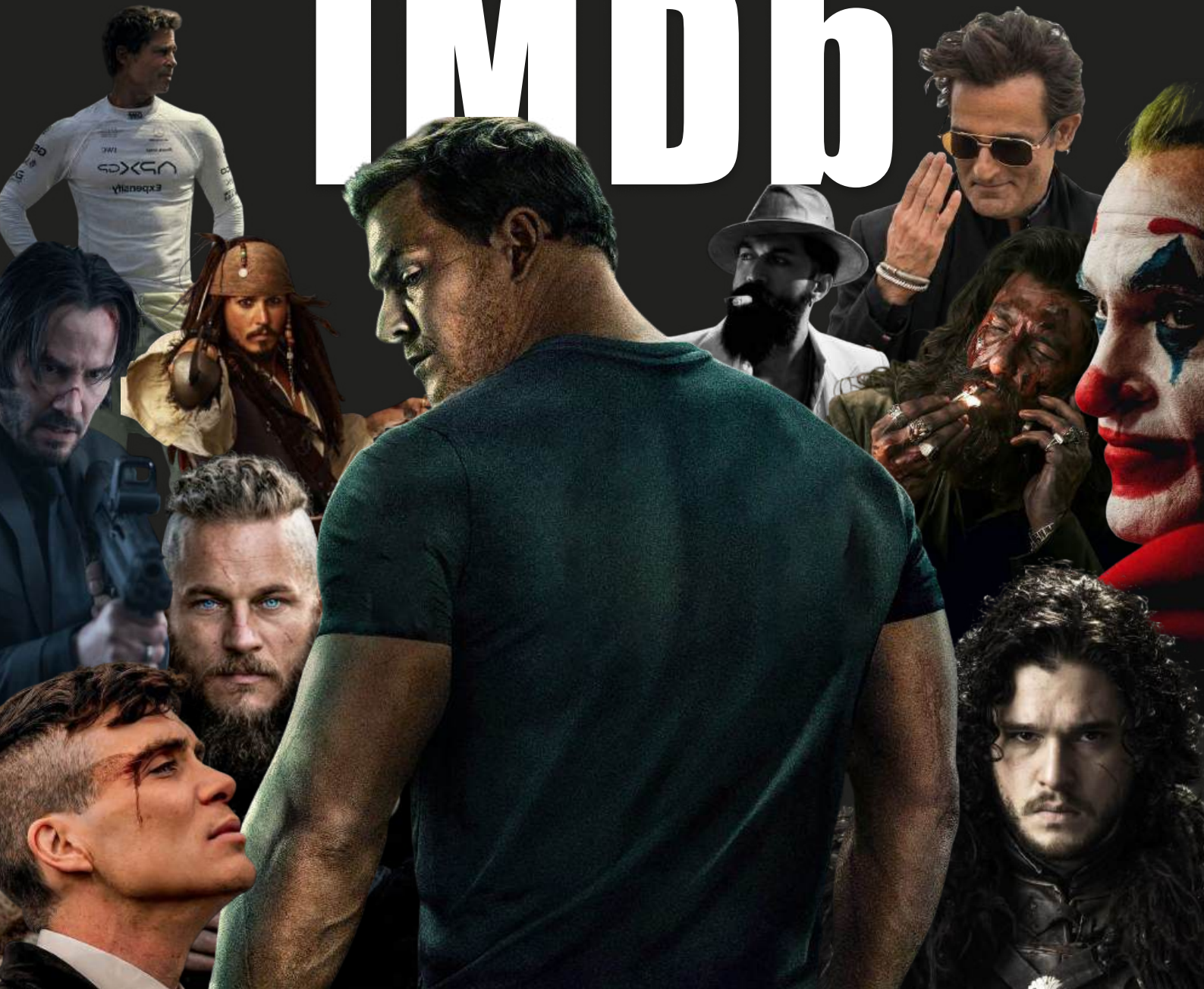


FROM RAW DATA TO BOX OFFICE DESTINY

IMDb



IMDb ANALYSIS

PROJECT REPORT

RATING PREDICTION

GENRE TREND ANALYSIS

RECOMMENDATION



01 MOVIES
SCRAPING

02 DATA
CLEANING



03 EDA &
VISUALIZATION

04 GENRE
EVOLUTION



05 RATING
PREDCITION

06 RECOMMEN-
DATION



PROJECT OBJECTIVE

"Decoding Cinema: Turning IMDb Metadata into Predictive Intelligence."

The primary objective of this project is to analyze movie data sourced from IMDb to uncover patterns and insights related to movie ratings, genres, popularity, and audience preferences.

Steps:

- Collecting structured movie data from multiple IMDb categories (Top Rated, Popular, etc.)
- Building a scalable web scraping pipeline
- Cleaning and organizing the data for analysis
- Preparing the dataset for further tasks like EDA, visualization, and modeling

Goals:

The goal is to create a reliable dataset, exploratory data analysis, and model training that can help answer questions such as:

- What factors influence high-rated movies?
- Which genres perform best over time?
- How do votes, ratings, and popularity correlate?
- Which movies you should prefer after watching a one?

MOVIES SCRAPING

The scraping task is designed to systematically extract movie from IMDb across multiple categories while ensuring scalability, efficiency, and data quality.

FEATURE SELECTION

Before scraping, we defined the key features required for analysis. These features were selected based on their analytical importance:

Basic Movie Information

- Movie URL (Unique Identifier)
- Movie Title
- Release Date
- Duration

Rating & Popularity Metrics

- IMDb Rating & Metascore
- Number of Votes
- User & Critic Reviews Count
- Watchlist Count

Content Attributes

- Genre(s)
- Description
- Keywords
- Director
- Cast (Top actors)
- Certificate/Content Rating (U/A, PG-13, etc.)

Financial Features

- Gross Revenue, Budget, & Opening Weekend

DATA SOURCE & CATEGORY SELECTION

Data is collected from different IMDb listing pages to prevent the biasness while training the model. So, dataset must be large enough, balanced, diverse, and not biased. That's why we scrape from the different listing pages such as:

- Top Rated Movies (8-10)
- Most Popular Movies (Popularity)
- Mid Rated Movies (5-8)
- Low Rated Movies (1-5)
- Old Movies (1980s-2000s)

Each category provides a list of URLs and by combining the movies from different categories will make the dataset richer.

So, the distribution of collection of movies from each category are:

- 1000 (Top Rated Movies)
- 1500 (Most Popular Movies)
- 2500 (Mid Rated Movies)
- 1200 (Low Rated Movies)
- 800 (Old Movies)

SCRAPING STRATEGY

The scraping process is divided into two main stages:

Stage 1: Collect Movie Links

- Extract movie URLs from listing pages
- Handle load more button using selenium
- Store all links in a CSV file

Stage 2: Scrape Movie Details

- Visit each movie page using selenium
- Extract selected features using beautiful soup
- Store structured data in CSV file

IMPLEMENTATION APPROACH

As we're scraping the IMDb website, it has anti-bots which will block our IP address if it finds the scraping behavior suspicious. So, instead of scraping all movies at once, a batch-wise approach along with short-time breaks to:

- **Avoid blocking from IMDb servers**
- **Handle failures safely**
- **Enable resume capability**

After each request, added a uniform time delay of 5-8s and after each batch, took 15-20 mins break to make it look-like human behavior, then saved the results in a csv file after each batch.

Libraries Used:

- **selenium** → (For fetching web pages)
- **BeautifulSoup** → (For parsing HTML)
- **pandas** → (To store structured data)
- **time** → (To add delays)
- **os** → (For file operations)

SCRAPING CODE

As we've divided scraping process is divided into stages:

- Collecting movie links
- Scraping movie details separately from the each movie page

Let's start with the collecting movie links with the help of 'Selenium' module to scrape movies links from different listing pages.

So, we'll scrape all these listing pages:

- Top Rated, Mid Rated, Low Rated
- Popular & Old Movies

Top Rated Movies (Quality Dataset):

Goal:

To extract the high-quality cinema dataset for Rating Prediction, Director success modeling, and quality benchmark.

URL:

```
https://www.imdb.com/search/title/?title_type=feature&num_votes=1000,&user_rating=8.0,10&release_date=1980-01-01,2024-12-31&sort=user_rating,desc
```

Most Popular Movies (Trend Dataset):

Goal:

To check the modern audience trends for Genre Trend Analysis and modern audience behavior.

URL:

```
https://www.imdb.com/search/title/?title_type=feature&release_date=2000-01-01,2024-12-31&sort=metascore,asc
```

Mid Rated Movies (Core ML Dataset):

Goal:

To extract larger dataset from this category because majority films lie here and is important for ML models for realistic rating prediction and balanced success prediction.

URL:

```
https://www.imdb.com/search/title/?title_type=feature&num_votes=5000,&user_rating=5.0,8.0&release_date=2000-01-01,2024-12-31&sort=num_votes,desc
```

Low Rated Movies (Failure Dataset):

Goal:

To teach model failure patterns because model cannot learn without failures which prevents biasness and improves success prediction.

URL:

```
https://www.imdb.com/search/title/?title_type=feature&num_votes=1000,&user_raing=1.0,5.5&release_date=2000-01-01,2024-12-31&sort=user_rating,asc
```

Old Movies (Trend Over Time Dataset):

Goal:

To extract old movies for genre evolution analysis and to see the historical cinema trends, and long-term rating dynamics.

URL:

```
https://www.imdb.com/search/title/?release_date=1980-01-01,1999-12-31&num_votes=10000,&sort=year,asc
```

Movies Link Collection:

Using selenium, scrape the movie links for these all different listing pages in different csv files.

Code:

```
import time
import random
import pandas as pd

from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager

from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.options import Options

#----- ANTI BOT SETTINGS -----

def get_driver():

    chrome_options = Options()
```



```

chrome_options.add_argument("--disable-blink-features=AutomationControlled")
chrome_options.add_argument("--start-maximized")
chrome_options.add_argument("--disable-infobars")
chrome_options.add_argument("--disable-extensions")

chrome_options.add_experimental_option("excludeSwitches",
                                       ["enable-automation"])
chrome_options.add_experimental_option('useAutomationExtension', False)

driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()),
                        options=chrome_options)

driver.execute_script("
Object.defineProperty(navigator, 'webdriver', {get: () => undefined})")

return driver

#----- SCROLL FUNCTION -----

def scroll_page(driver, scrolls=20):
    for i in range(scrolls):
        print(f"Scrolling {i+1}")
        driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
        time.sleep(random.uniform(2,4))

        try:
            load_more = driver.find_element(By.XPATH,
                                           ' //button//span[text()="50 more"]')

            load_more.click()
            print("Clicked Load More")
            time.sleep(random.uniform(3,5))
        except:
            print("No Load More Button")
            break

#----- EXTRACT MOVIE LINKS -----

def extract_links(driver):
    movies = driver.find_elements(By.XPATH, ' //a[contains(@href, "/title/")]')
    links = []

```

```

for m in movies:
    link = m.get_attribute("href")

    if "/title/" in link and "ref_" in link:
        clean = link.split("?")[0]
        links.append(clean)

links = list(set(links))

return links

#----- MAIN SCRAPER -----

def main():

    driver = get_driver()

    url = "https://www.imdb.com/search/title/?title_type=feature&
release_date=1980-01-01,1999-12-31&num_votes=10000,&sort=year,asc"

    driver.get(url)
    time.sleep(5)
    scroll_page(driver, scrolls=15)

    links = extract_links(driver)

    df = pd.DataFrame({"movie_url": links})
    df['Scrape_Category'] = 'Old Movies'

    df.to_csv("raw_data/Old_Movies_800.csv", index=False)
    print(f"Total Movies Collected: {len(links)}")

    driver.quit()

if __name__ == "__main__":
    main()

```

Output:

- Run the same script for different listing pages by changing the URL and 'Scrape Category'
- Extract movie links for each category and store in different CSV files.

Movies Data Collection:

Using selenium and BeautifulSoup, we'll scrape the each movie separately extracting all the required features for the dataset.

Code:

```
import time
import random
import json
import re
import pandas as pd
import glob

from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
from selenium.webdriver.chrome.options import Options

from bs4 import BeautifulSoup

#----- DRIVER SETUP -----

def setup_driver():
    options = Options()

    options.add_argument("--start-maximized")
    options.add_argument("--disable-blink-features=AutomationControlled")
    options.add_argument("--disable-infobars")
    options.add_argument("--disable-extensions")

    driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()),
                             options=options)

    return driver
```

```

#----- EXTRACT JSON -----

def extract_json_ld(soup):
    scripts = soup.select('script[type="application/ld+json"]')

    for sc in scripts:
        try:
            data = json.loads(sc.string)
            if "aggregateRating" in data:
                return data
        except:
            continue

    return None

#----- MINUTE CONVERSION -----

def convert_iso_duration(duration):
    hours = re.search(r'(\d+)H', duration)
    minutes = re.search(r'(\d+)M', duration)

    total = 0
    if hours:
        total += int(hours.group(1)) * 60
    if minutes:
        total += int(minutes.group(1))

    return total if total > 0 else None

#----- EXTRACT DETAILS -----

def scrape_movie(driver, url):
    try:
        driver.get(url)

        # WAIT like human
        time.sleep(random.uniform(3, 5))

        soup = BeautifulSoup(driver.page_source, "html.parser")
        json_data = extract_json_ld(soup)

```

```

if not json_data:
    print("JSON missing:", url)
    return None

data = {}

# BASIC INFO
data['title'] = json_data.get('name')
data['genres'] = "|".join(json_data.get('genre', []))
data['release_date'] = json_data.get('datePublished')

runtime_iso = json_data.get('duration')
data['duration'] = convert_iso_duration(runtime_iso)

rating = json_data.get('aggregateRating', {})
data['rating'] = rating.get('ratingValue')
data['votes'] = rating.get('ratingCount')
data['content_rating'] = json_data.get('contentRating')

# ACTORS
actors = json_data.get('actor', [])
data['star_1'] = actors[0]['name'] if len(actors) > 0 else None
data['star_2'] = actors[1]['name'] if len(actors) > 1 else None
data['star_3'] = actors[2]['name'] if len(actors) > 2 else None

# DIRECTOR
directors = json_data.get('director', [])
data['director'] = directors[0]['name'] if directors else None

# Countries
try:
    country_block = soup.select_one('li[data-testid=
                                     "title-details-origin"]')

    if country_block:
        countries = country_block.select('a')
        data['countries'] = '|'.join([c.text.strip() for c in countries])
except:
    data['countries'] = None

# Languages
try:
    language_block = soup.select_one('li[data-testid=
                                     "title-details-languages"]')

```

```

        if language_block:
            languages = language_block.select('a')
            data['languages'] = '|'.join([l.text.strip() for l in languages])
    except:
        data['languages'] = None

# Watchlist Count
try:
    watchlist = soup.select_one('div[data-testid="tm-box-wl-count"]')
    if watchlist:
        text = watchlist.text.strip()
        # Example: "Added by 646K users"
        data['watchlist_count'] = text.replace("Added by", "")
                                .replace("users", "").strip()
except:
    data['watchlist_count'] = None

# Reviews Section (User, Critic, Metascore)
try:
    review_blocks = soup.select('ul[data-testid="reviewContent-all-reviews"] li')

    for block in review_blocks:
        label = block.select_one('.label')
        score = block.select_one('.score')

        if label and score:
            label_text = label.text.strip().lower()
            score_text = score.text.strip()

            if "user reviews" in label_text:
                data['user_reviews'] = score_text

            elif "critic reviews" in label_text:
                data['critic_reviews'] = score_text

            elif "metascore" in label_text:
                data['metascore'] = score_text
except:
    pass

# FINANCIALS
try:
    data['gross'] = soup.select_one('[data-testid="title-boxoffice-cumulativeworldwidegross"] div').text.strip()

```



```

except:
    data['gross'] = None

try:
    data['budget'] = soup.select_one('[data-testid="title-boxoffice-
        budget"] div').text.strip()
except:
    data['budget'] = None

try:
    block = soup.find("li", {"data-testid":
        "title-boxoffice-openingweekenddomestic"})
    data['opening_weekend'] = block.find("span",
        class_="ipc-metadata-list-item__list-content-item").text.strip()
except:
    data['opening_weekend'] = None

return data

except Exception as e:
    print("Error:", e, url)
    return None

#----- COMBINED LINKS DATASET -----

files = glob.glob("raw_data/*.csv")

df_links = pd.concat([pd.read_csv(f) for f in files])

# Join same movies with multiple categories
df_links = df_links.groupby('movie_url').agg({
    'Scrape_Category': lambda x: '|'.join(set(x))
}).reset_index()

df_links = df_links.iloc[1:].reset_index(drop=True)

#----- BATCH-WISE SCRAPING -----

def run_batch_scraper(batch_size=300):
    df = df_links
    output_file = "IMDb_movies_data.csv"

```

```

# Resume logic
if os.path.exists(output_file):
    existing_df = pd.read_csv(output_file)
    start_index = len(existing_df)
    print(f"Resuming from index {start_index}")
else:
    start_index = 0
    print("Starting fresh scraping")

end_index = min(start_index + batch_size, len(df))
driver = setup_driver()

for i in range(start_index, end_index):

    url = df.loc[i, "movie_url"]
    print(f"Scraping {i+1}/{len(df)}")
    movie = scrape_movie(driver, url)

    if movie:
        movie['movie_url'] = url

        pd.DataFrame([movie]).to_csv(
            output_file,
            mode='a',
            header=not os.path.exists(output_file),
            index=False
        )
        time.sleep(random.uniform(2,4))

driver.quit()
print(f"Batch completed: {start_index} → {end_index}")

#Run the scraper
run_batch_scraper(batch_size=300)

```

Output:

- Stores the movie details of all combined categories in a single CSV file.
- Now, using that CSV file, we'll move on to the next phase of data cleaning and preprocessing to make data cleaner or richer for the exploratory data analysis and visualization.

DATASET SCHEMA

The final dataset after scraping all the movies data will look like:

Columns	Description
movie_url	URL or unique identifier for movies
title	name of the movie
genres	movie's different genre types
release_date	date when the movie is released
duration	total runtime of the movie
rating	rating of a movie out of 10
votes	count of the ratings
content_rating	certificate of a movie (eg. R rated)
star_1, star_2, star_3	name of the Top 3 actors
countries	country of origin
languages	release languages
director	name of the director
gross	total box office collection
budget	total budget of the movie (estimated)
metascore	weighted score of a movie by IMDb
user_reviews	count of user reviews
critic_reviews	count of critic reviews
description	Description about the movie
keywords	keywords for the movie
watchlist_count	count of people added as a watchlist

DATA CLEANING

INTRODUCTION

After collecting raw movie data from IMDb, the dataset contained several inconsistencies such as duplicate entries, missing values, inconsistent formats, and irrelevant attributes. Data cleaning was therefore an essential step to ensure the dataset was accurate, consistent, and suitable for further analysis.

All preprocessing tasks were performed using Python pandas library to ensure reproducibility and efficient handling of large datasets.

REMOVAL OF DUPLICATE RECORDS

Since movies were scraped from multiple categories (e.g., Top Rated, Popular, etc.), the same movie appeared multiple times with different category labels, also we've already joined the different category before scraping each movie data. But, still we'll check for the duplicate records.

To address this:

- Duplicate entries were identified using the unique movie URL
- All category labels corresponding to the same movie were merged into a single field
- Each movie was retained as a single record

This ensured that:

- No data was lost
- Category information was preserved
- Redundant rows were eliminated

Code:

To check for duplicate values, we'll use the 'duplicated()' function. And to join the multiple categories we'll use the 'groupby()' with 'join()' function.

```
# Importing basic libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load movies data
df = pd.read_csv("IMDb_movies_data.csv")
df.head()
print("Before dropping: ",df.shape)

# Finding duplicate values
dup_url = df['movie_url'].duplicated().sum()
print("Total Duplicate URL Rows: ",dup_url)

# Dropping duplicate values
df = df.drop_duplicates(subset=['movie_url'])
print("Before dropping: ",df.shape)
```

Output:

- All the duplicate rows with same movie url has been dropped.

Output:

Before dropping: (5645, 24)

Total Duplicate URL Rows: 9

After dropping: (5636, 24)

Handling Missing Values:

Missing values were handled based on the importance of each feature and the proportion of missing data.

Before deciding we'll check the percentage (%) of the missing values for each column:

Rule:

- < 20% missing → Keep
- 20–40% → Use carefully
- 40% & above → Drop Column

Code:

```
df.isna().sum()
df.isna().mean() * 100
```

Output:

Output:

title	0	0.000000
genres	7	0.124202
description	6	0.106458
keywords	164	2.909865
release_date	76	1.348474
duration	0	0.000000
content_rating	1010	17.920511
director	2	0.035486
star_1	6	0.106458
star_2	35	0.621008
star_3	47	0.833925
countries	0	0.000000
languages	42	0.745209
rating	0	0.000000
metascore	1988	35.273243
votes	0	0.000000
user_reviews	109	1.933996
critic_reviews	450	7.984386
watchlist_count	623	11.053939
gross	1390	24.662881
budget	1784	31.653655
opening_weekend	2227	39.513840
movie_url	0	0.000000
dtype:	float64	

I. Genres

A small portion of records had missing genres due to incomplete information about genres on the movie page.

Handling Approach:

- Missing entries were replaced with “Unknown”

This allowed column to remain usable for categorical analysis without discarding data.

Code:

```
df['genres'] = df['genres'].fillna('Unknown')
df['genres'].isna().sum()
```

Output:

```
Output:
np.int64(0)
```

II. Release Date

A small portion of records had missing release dates due to incomplete JSON data. However, month and year information was available on the webpage.

Handling Approach:

- Missing entries were re-scraped from the source
- The first day of the respective month was assigned (i.e., day=1)
- Default date was assigned as ‘2000-01-01’ for null records even after re-scraping. We got 7 records out of 5649, so replaced by default.
- All dates were converted into a standard datetime format

This approach ensured maximum data completeness while maintaining consistency.

Code:

```
import re, time, random
from datetime import datetime
from IMDB_Scraping import get_driver
from bs4 import BeautifulSoup

print("Total null date records: ",df['release_date'].isna().sum())
missing_df = df[df['release_date'].isna()]

#----- Date Conversion -----

def create_date(month, year):
    try:
        return datetime.strptime(f"1 {month} {year}", "%d %B %Y").date()
    except:
        return None

#----- Scrape Month & Year -----

def scrape_month_year(driver, url):
    driver.get(url)

    time.sleep(random.uniform(3, 5))

    soup = BeautifulSoup(driver.page_source, "html.parser")

    try:
        block = soup.find("div", {"data-testid": "title-details-section"})
        date_text = block.select_one(
            '[data-testid="title-details-releasedate"] div').text.strip()
    except:
        date_text = '2000-01-01'

    match = re.match(r'(.*\d).*', date_text)

    try:
        month, year = match.group(1).split(' ')
    except:
        year = match.group(1)
        month = 'January'

    return month, year
```



```
#----- Re-Scrape Missing Records -----

driver = get_driver()
count = 1

for idx, row in missing_df.iterrows():
    url = row['movie_url']
    print(f"Re-Scraping: {count}/{len(missing_df)}")
    month, year = scrape_month_year(driver, url)
    df.loc[idx, "release_date"] = create_date(month, year)

    time.sleep(random.uniform(2,4))
    count += 1

driver.quit()

print("Total null records after Re-Scraping: ",df['release_date'].isna().sum())

# Fill default date for null records even after re-scraping
df['release_date'] = df['release_date'].fillna('2000-01-01')

# Convert to datetime datatype
df["release_date"] = pd.to_datetime(df["release_date"], errors="coerce")
print("Datatype: ",df['release_date'].dtype)
```

Output:

Output:

```
Total Null Records: 76
Re-Scraping: 1/76
Re-Scraping: 2/76
Re-Scraping: 3/76
Re-Scraping: 4/76
.
.
Total null records after Re-Scraping: 7

Datatype: dtype('<M8[ns]') → datetime
```

III. Description

A small portion of records had missing description due to incomplete information about it on the movie page.

Handling Approach:

- Missing entries were replaced with “No Description available”

This approach allowed column to remain usable for categorical analysis without discarding data.

Code:

```
df['description'] = df['description'].fillna('No Description available')  
df['description'].isna().sum()
```

IV. Keywords

A small portion of records had missing keywords due to incomplete information about it on the movie page.

Handling Approach:

- Missing entries were replaced with “Empty String” for NLP Tasks

This approach allowed column to remain usable for categorical analysis without discarding data.

Code:

```
df['keywords'] = df['keywords'].fillna("")  
df['keywords'].isna().sum()
```

Output:

```
Output:  
np.int64(0)  
np.int64(0)
```

V. Content Rating

Approximately ~18% of missing values were observed in the content rating column due to incomplete information about content rating on the movie page.

Handling Approach:

- Missing entries were replaced with “Unknown”

This approach allowed column to remain usable for categorical analysis without discarding data.

Code:

```
print("Unique Content Rating Values: \n",df['content_rating'].unique())
print("Total null content rating records: ",df['content_rating'].isna().sum())

# Filling null records with unknown
df['content_rating'] = df['content_rating'].fillna('Unknown')

print("Total null records after filling: ",df['content_rating'].isna().sum())
```

Output:

Output:

```
Unique Content Rating Values:
array(['PG-13', nan, 'Unrated', 'R', 'PG', 'Not Rated', 'G',
      'TV-MA', 'NC-17', 'TV-PG', 'TV-14', '16+', 'X', '13+', 'TV-G',
      'Approved', 'TV-Y7', '18+', 'TV-13'], dtype=object)

Total null content rating records: 1016
Total null records after filling: 0
```

VI. Star & Director

Less than 1% of missing values were observed in the star_1, star_2, star_3, and director columns due to incomplete information about stars and director on the movie page.

Handling Approach:

- Missing entries were replaced with “Unknown”

This approach allowed column to remain usable for categorical analysis without discarding data.

Code:

```
print("Total null records in star & director columns: ")
df[['star_1', 'star_2', 'star_3', 'director']].isna().sum()

# Filling null records with unknown
df['star_1'] = df['star_1'].fillna('Unknown')
df['star_2'] = df['star_2'].fillna('Unknown')
df['star_3'] = df['star_3'].fillna('Unknown')
df['director'] = df['director'].fillna('Unknown')

print("Total null records after filling: ")
df[['star_1', 'star_2', 'star_3', 'director']].isna().sum()
```

Output:

Output:

Total null records in star & director columns:

star_1	7
star_2	36
star_3	48
director	3

Total null records after filling:

star_1	0
star_2	0
star_3	0
director	0

VII. Watchlist Count & Reivews

A small portion of records had missing values in watchlist_count, user_reviews, and critic_reviews due to incomplete information about it on the movie page.

Handling Approach:

- Missing entries were replaced with “0”, because as it was not present which means no one had given the reviews nor added to the watchlist.
- Values in “K” and “M” are converted into numeric form (1000s and 1000000s).

Code:

```
def convert_to_number(x):
    if pd.isna(x):
        return 0

    if isinstance(x, (int, float)):
        return x

    x = str(x).replace(',', '').strip().upper()
    if 'K' in x:
        return float(x.replace('K', '')) * 1_000
    elif 'M' in x:
        return float(x.replace('M', '')) * 1_000_000
    else:
        return float(x)

df['watchlist_count'] = df['watchlist_count'].apply(convert_to_number)
df['user_reviews'] = df['user_reviews'].apply(convert_to_number)
df['critic_reviews'] = df['critic_reviews'].apply(convert_to_number)

df[['user_reviews', 'critic_reviews', 'watchlist_count']].isna().sum()
```

Output:

Output:

user_reviews	0
critic_reviews	0
watchlist_count	0

VIII. Opening Weekend

The opening_weekend column had approximately 40% missing values. Additionally, it was not directly aligned with the project's focus on ratings and audience behavior.

Handling Approach:

- This column was removed from the dataset.

This helped reduce noise and simplify further analysis.

Code:

```
# Dropping opening_weekend columns
df = df.drop(columns=["opening_weekend"])
```

CURRENCY CONVERSION

Distinct currency formats present in financial columns such as gross and budget were identified by extracting currency symbols and codes.

Handling Approach:

- Build conversion mapping to convert into one common currency (in our case USD).

This approach enabled proper standardization and conversion of monetary values into a common currency.

Code:

```
import re

# Converting Object to String
df["gross"] = df["gross"].astype(str)
df["budget"] = df["budget"].astype(str)
```

```
#----- EXTRACT UNIQUE CURRENCY TYPES -----

def extract_currency_type(x):
    if pd.isnull(x):
        return None

    # Extract currency prefix (before numbers)
    match = re.match(r"^[^\d]+", x)
    return match.group().strip() if match else None

df["currency_type"] = df["budget"].apply(extract_currency_type)
print("Unique Currencies: ", df['currency_type'].unique())

#----- CONVERSION MAPPING -----

conversion_rates = {
    "$": 1.0,          # USD
    "£": 1.25,         # GBP
    "€": 1.10,         # Euro
    "¥": 0.007,        # Japanese Yen
    "CN¥": 0.14,       # Chinese Yuan
    "₹": 0.012,        # Indian Rupee
    "A$": 0.65,        # Australian Dollar
    "CA$": 0.75,       # Canadian Dollar
    "NZ$": 0.60,       # New Zealand Dollar
    "R$": 0.20,        # Brazilian Real
    "MX$": 0.058,      # Mexican Peso
    "NT$": 0.031,      # Taiwan Dollar
    "₩": 0.00075,      # South Korean Won
    "SEK": 0.095,      # Swedish Krona
    "NOK": 0.095,      # Norwegian Krone
    "DKK": 0.15,       # Danish Krone
    "PLN": 0.25,       # Polish Zloty
    "HUF": 0.0028,     # Hungarian Forint
    "RON": 0.22,       # Romanian Leu
    "ZAR": 0.055,      # South African Rand
    "SAR": 0.27,       # Saudi Riyal
    "EGP": 0.032,      # Egyptian Pound
    "PKR": 0.0036,     # Pakistani Rupee
    "BDT": 0.0091,     # Bangladeshi Taka
    "NPR": 0.0075,     # Nepali Rupee
    "LKR": 0.0031,     # Sri Lankan Rupee
    "IDR": 0.000065,   # Indonesian Rupiah
    "MVR": 0.065,      # Maldivian Rufiyaa

```

```

# Old currencies (approx historical conversion)
"ITL": 0.00055, # Italian Lira
"FRF": 0.18,   # French Franc
"DEM": 0.55,   # German Mark
"TRL": 0.000000035, # Old Turkish Lira
"ATS": 0.073,  # Austrian Schilling
"RUR": 0.013,  # Old Russian Ruble

# Missing / unknown
"nan": None
}

#----- EXTRACT CURRENCY & AMOUNT -----

def extract_currency_amount(x):
    if pd.isnull(x):
        return None, None

    currency = re.match(r"^\d+", x)
    currency = currency.group().strip() if currency else None

    amount = re.findall(r"\d,+", x)
    amount = int(amount[0].replace(",", "")) if amount else None

    return currency, amount

#----- CONVERSION MAPPING -----

def convert_to_usd(x):
    currency, amount = extract_currency_amount(x)

    if currency in conversion_rates and amount:
        rate = conversion_rates[currency]
        if rate:
            return amount * rate

    return None

df['gross_usd'] = df['gross'].apply(convert_to_usd)
df["budget_usd"] = df["budget"].apply(convert_to_usd)

df.isna().sum()
df.dtypes

```


Output:

Output:

```
Unique Currencies: array(['$', 'nan', '£', 'A$', 'CA$', 'ITL',  
'DEM', 'FRF', '¥', 'SEK', 'NZ$', '₹', '€', 'TRL', 'ATS', 'PLN',  
'R$', 'CN¥', '₩', 'EGP', 'RON', 'RUR', 'SAR', 'IRR', 'BDT',  
'NOK', 'NT$', 'HUF', 'DKK', 'MX$', 'NPR', 'PKR', 'ZAR', 'LKR',  
'IDR', 'MVR'], dtype=object)
```

title	0	object
genres	0	object
release_date	0	datetime64[ns]
duration	0	int64
rating	0	float64
votes	0	int64
content_rating	0	object
star_1	0	object
star_2	0	object
star_3	0	object
director	0	object
gross	0	float64
budget	0	float64
movie_url	0	object
currency_type	0	object
gross_usd	1392	float64
budget_usd	1789	float64
dtype:	float64	

DATA TYPE CONVERSION

Several columns required conversion into appropriate data types. But, as we've extracted some numeric features such as duration, votes, and rating from the JSON, it didn't required the conversion.

However, release_date, user_reviews, critic_reviews, watchlist_count, gross, and budget are converted to datetime and numeric data type respectively in the previous task while handling currency conversions.

TEXT CLEANING & STANDARDIZATION

Text-based columns such as title, genres, content_rating, stars, and directors were cleaned by:

- Removing extra spaces & Standardized formatting.
- Title, Stars, and Director columns contains HTML encoded characters such as “&”, “'”, and “"”, these character are:

&apm;	→	&
'	→	'
"	→	“

- Using “unescape()” function of html module we can convert the all HTML entities.

This improved readability and ensured proper grouping during analysis.

Code:

```
import html

# Converting HTML entities
df['title'] = df['title'].apply(html.unescape)
df['star_1'] = df['star_1'].apply(html.unescape)
df['star_1'] = df['star_1'].apply(html.unescape)
df['star_1'] = df['star_1'].apply(html.unescape)
df['director'] = df['director'].apply(html.unescape)

# Stripping the text columns
df['title'] = df['title'].str.strip()
df['genres'] = df['genres'].str.strip()
df['content_rating'] = df['content_rating'].str.strip()
df['star_1'] = df['star_1'].str.strip()
df['star_2'] = df['star_2'].str.strip()
df['star_3'] = df['star_3'].str.strip()
df['director'] = df['director'].str.strip()

# Saving Cleaned dataset in CSV file
df.to_csv("data/IMDb_movies_cleaned.csv", index=False, encoding="utf-8")
```

EDA & VISUALIZATION

INTRODUCTION

Exploratory Data Analysis (EDA) was performed on the cleaned dataset obtained from IMDb to understand the underlying pattern, relationships, and distribution of variables.

The objectives of EDA were:

- To analyze trends in movie ratings and genres.
- To identify factors influencing movie success.
- To detect patterns useful for predictive modeling.
- To validate data quality and assumptions.

DATASET OVERVIEW

Let's start the EDA by exploration of the dataset which tells:

- Total number of movies
- Total number of features
- Data types of each column
- Summary Statistics for the numerical columns

Code:

```
df = pd.read_csv('data/IMDb_movies_cleaned.csv')
# Total no. of records and features
df.shape
# Datatype and Non-Null Count
df.info()
# Summary Statistics
df.describe()
```

Output:

Output:

(5636, 26)

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5636 entries, 0 to 5635
Data columns (total 26 columns):
#   Column                Non-Null Count  Dtype
---  -
0   title                  5636 non-null   object
1   genres                 5636 non-null   object
2   description            5636 non-null   object
3   keywords               5472 non-null   object
4   release_date           5636 non-null   object
5   duration               5636 non-null   int64
6   content_rating         5636 non-null   object
7   director               5636 non-null   object
8   star_1                 5636 non-null   object
9   star_2                 5636 non-null   object
10  star_3                 5636 non-null   object
11  countries              5636 non-null   object
12  languages              5636 non-null   object
13  rating                 5636 non-null   float64
14  metascore              3648 non-null   float64
15  votes                  5636 non-null   int64
16  user_reviews           5636 non-null   float64
17  critic_reviews         5636 non-null   float64
18  watchlist_count        5636 non-null   float64
19  gross                  4246 non-null   object
20  budget                 3852 non-null   object
21  movie_url              5636 non-null   object
22  category               5636 non-null   object
23  currency_type          3852 non-null   object
24  gross_usd              4246 non-null   float64
25  budget_usd             3849 non-null   float64
dtypes: float64(7), int64(2), object(17)
```

	duration	rating	votes	gross_usd	budget_usd
count	5639.000000	5639.000000	5.639000e+03	4.247000e+03	3.850000e+03
mean	111.247384	6.180351	1.369148e+05	1.202507e+08	4.054147e+07
std	27.293525	1.902076	2.390249e+05	2.201389e+08	5.287713e+07
min	47.000000	1.000000	7.000000e+01	3.900000e+01	7.000000e-08
25%	94.000000	5.500000	4.594000e+03	5.484066e+06	5.000000e+06
50%	106.000000	6.700000	6.875000e+04	3.928099e+07	2.000000e+07
75%	123.000000	7.600000	1.580205e+05	1.305330e+08	5.400000e+07
max	607.000000	9.400000	3.169375e+06	2.923711e+09	4.149000e+08

UNIVARIATE ANALYSIS

Univariate analysis focuses on examining each variable independently to understand its distribution, central tendency, and variability. The objective is to:

- Identify the overall distribution of key features.
- Detect skewness, outliers, and anomalies.
- Understand frequency patterns in categorical data
- Gain initial insights before performing deeper analysis

This step helps in building intuition about the dataset and guides further analysis and feature selection.

I. Distribution of IMDb Ratings

Why this analysis was performed?

To understand how movie ratings are distributed and identify common rating ranges across the dataset.

Code:

```
import matplotlib.pyplot as plt
import seaborn as sns

# Rating Distribution using Histogram
plt.figure(figsize=(10,6))

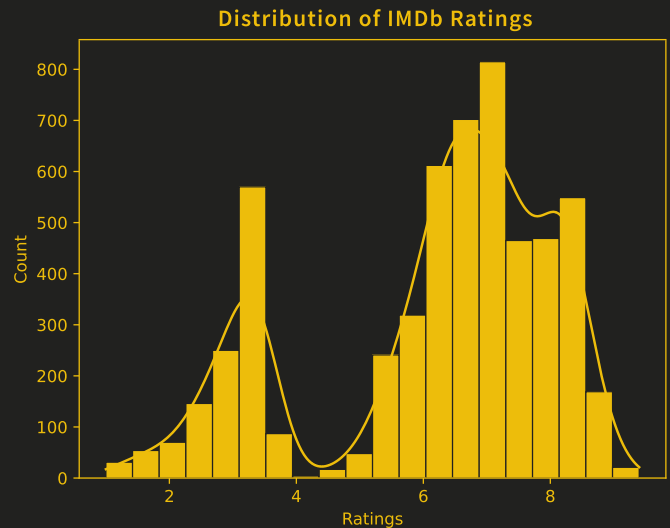
sns.histplot(df['rating'], bins=20, kde=True)
plt.title('Distribution of IMDb Ratings')
plt.xlabel('Ratings')
plt.ylabel('Count')

# For saving the plot as a image
plt.savefig("images/ratings_distribution.png", dpi=300)

plt.show()
```

Insights:

- Ratings are mostly concentrated between 6 and 8.
 - Very few movies have ratings below 4 or above 9.
 - The distribution appears slightly left-skewed (negatively skewed)
 - Most movies receive moderately high ratings, indicating a bias toward well-rated films in the dataset.
- Extremely low-rated movies are rare
 - This distribution suggests that ratings alone may have limited variance for prediction and may require additional features



II. Distribution of Votes

Why this analysis was performed?

To analyze the popularity of movies based on the number of user votes.

Code:

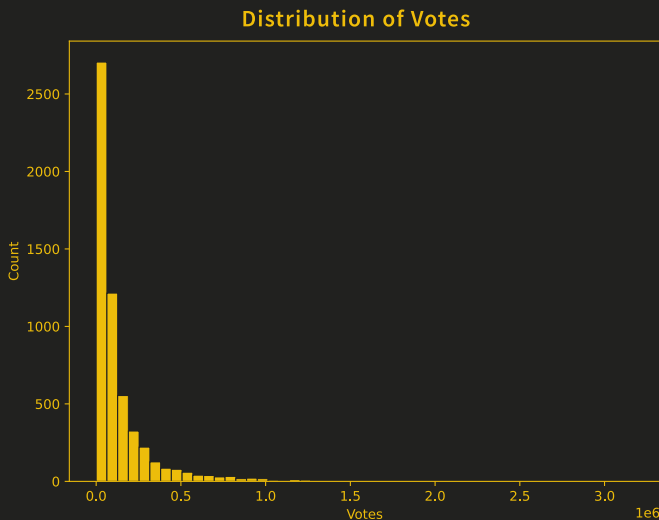
```
# Votes Distribution using Histogram
plt.figure(figsize=(8,6))

sns.histplot(df['votes'], bins=50)
plt.title('Distribution of Votes')
plt.xlabel('Votes')
plt.ylabel('Count')

# For saving the plot as a image
plt.savefig("images/votes_distribution.png", dpi=300)

plt.show()
```

Insights:



- The distribution is highly right-skewed.
- A large number of movies have low vote counts.
- A small number of movies have extremely high votes.
- Popularity is unevenly distributed across movies.

- Few blockbuster movies dominate in terms of audience engagement.
- This suggests that vote count is an important feature but may require transformation (e.g., log scaling) for further analysis.

III. Number of Movies per Year

Why this analysis was performed?

To analyze trends in movie production over time.

Code:

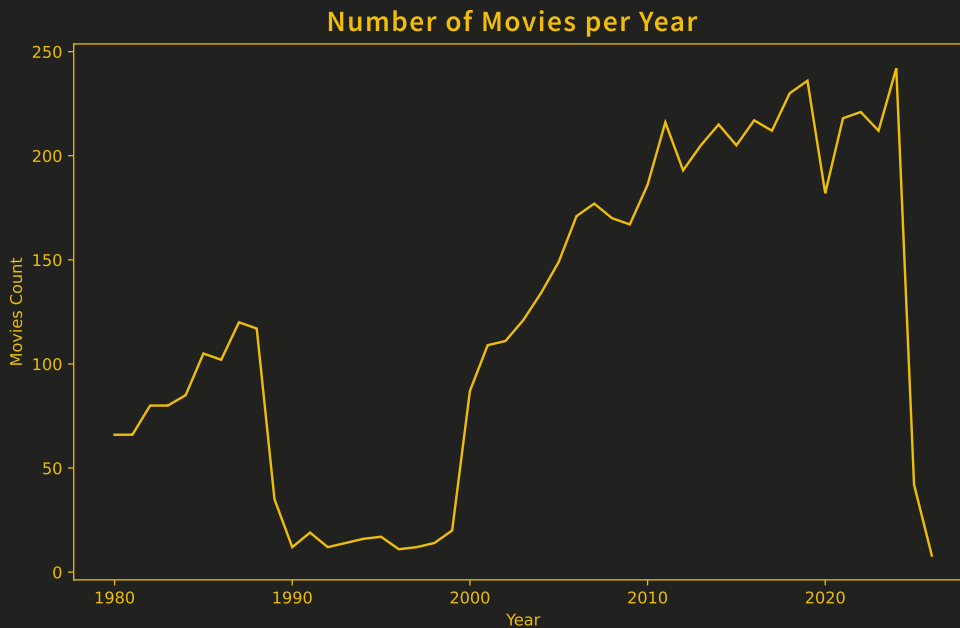
```
# Movies per year using Line Chart
df['release_year'] = df['release_date'].dt.year
movies_per_year = df['release_year'].value_counts().sort_index()

plt.figure(figsize=(8,6))

plt.plot(movies_per_year)
plt.title('Number of Movies per Year')
plt.xlabel('Year')
plt.ylabel('Movies Count')

plt.show()
```

Insights:



- A general increasing trend in the number of movies over the years, indicates movies production has grown significantly over time.
- Noticeable dip during the early 1990s.
- Significant growth after the 2000s.
- Sharp decline in the most recent years (most likely due to incomplete data).
- Time-based trends can be useful for understanding industry growth and temporal patterns

IV. Top Genres Distribution

Why this analysis was performed?

To identify the most frequently occurring genres in the dataset.

Before performing genre analysis, first we need to unpivot the genres as there are multiple genres in one movie (e.g., Comedy|Drama|Romance).

- With the help of '`split()`' function and assign separator "|", to split the genres into list.
- After splitting, using the '`explode()`' function, we can unpivot the genres.

Code:

```
# Splitting Genre Column
df_genre = df.copy()
df_genre['genres'] = df_genre['genres'].str.split("|")

# Unpivoting Genre Column
df_genre = df_genre.explode('genres')

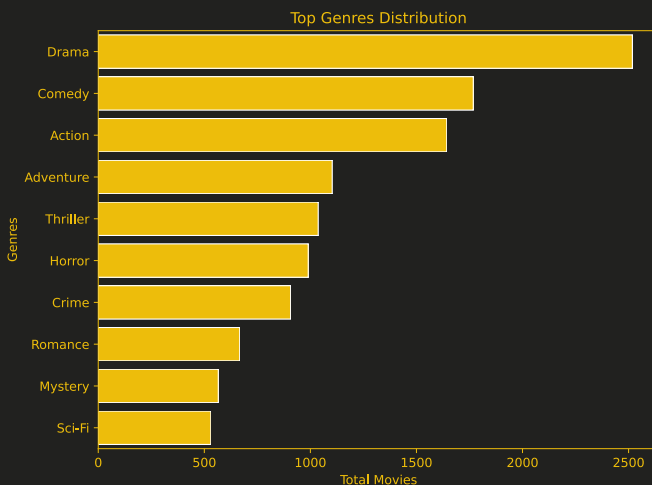
top_genres = df_genre['genres'].value_counts().head(10)

# Genre Distribution using Bar Plot
plt.figure(figsize=(8,6))

sns.barplot(x=top_genres.values, y=top_genres.index)

plt.title("Top Genres Distribution")
plt.xlabel('Total Movies')
plt.ylabel('Genres')
plt.show()
```

Insights:



- Drama is the most dominant genre.
- Followed by Comedy and Action.
- Genres like Sci-Fi and Mystery appear less frequently.
- The dataset is genre-imbalanced, with certain genres dominating.

- Popular genres like Drama and Comedy may influence overall trends.
- Less frequent genres may require careful handling during analysis and modeling

BIVARIATE ANALYSIS

Bivariate analysis examines the relationship between two variables to identify patterns, correlations, and dependencies. The objective is to:

- Understand how one variable influence another.
- Detect positive, negative, or no correlation.
- Identify trends useful for prediction and decision making.
- Support feature selection for model building.

This step provided deeper insights beyond individual variable analysis and helps uncover meaningful relationships within the dataset.

I. Ratings vs Votes

Why this analysis was performed?

To examine the relationship between movie ratings and audience engagement which is measured by votes.

Code:

```
# Rating vs Votes using Scatterplot
plt.figure(figsize=(10,6))

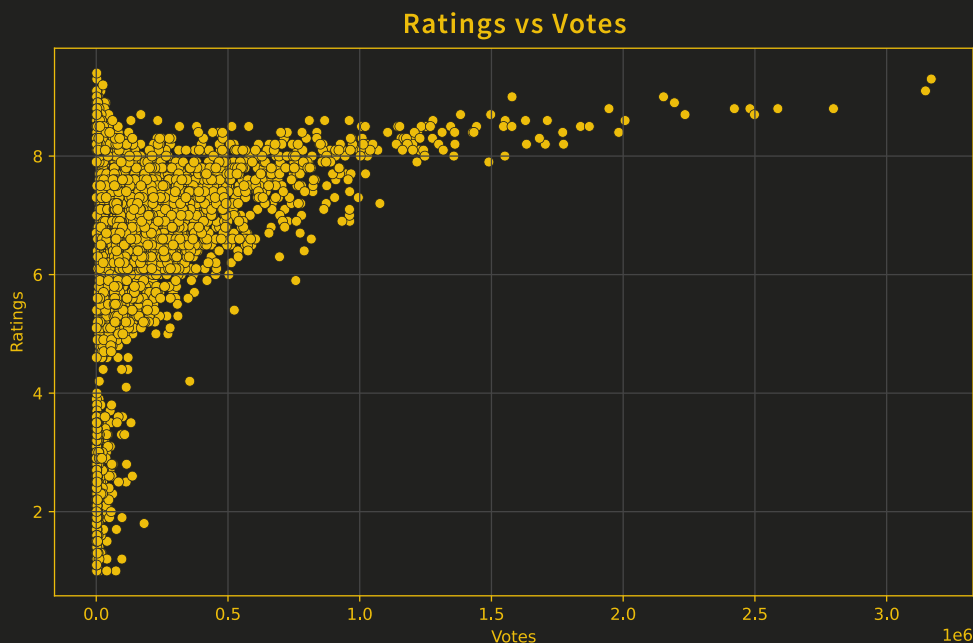
sns.scatterplot(x="votes", y="rating", data=df)
plt.title('Ratings vs Votes')
plt.xlabel('Votes')
plt.ylabel('Ratings')

plt.grid()

# For saving the plot as a image
plt.savefig("images/rating_vs_votes.png", dpi=300)

plt.show()
```

Insights:



- Slight upward trend between votes and ratings, which tells there is a weak positive correlation between ratings and votes.
- Most movies are clustered in the lower vote range.
- Popular movies tend to have better ratings, as high-vote movies generally have ratings above 7.
- However, high ratings are also present in low-vote movies, indicating that popularity is not the only factor.

II. Average Rating by Genre

Why this analysis was performed?

To identify how ratings vary across different movie genres.

Code:

```
# Average Rating by Genre using Bar Plot
genre_rating = df_genre.groupby('genres')['rating'].mean().sort_values()

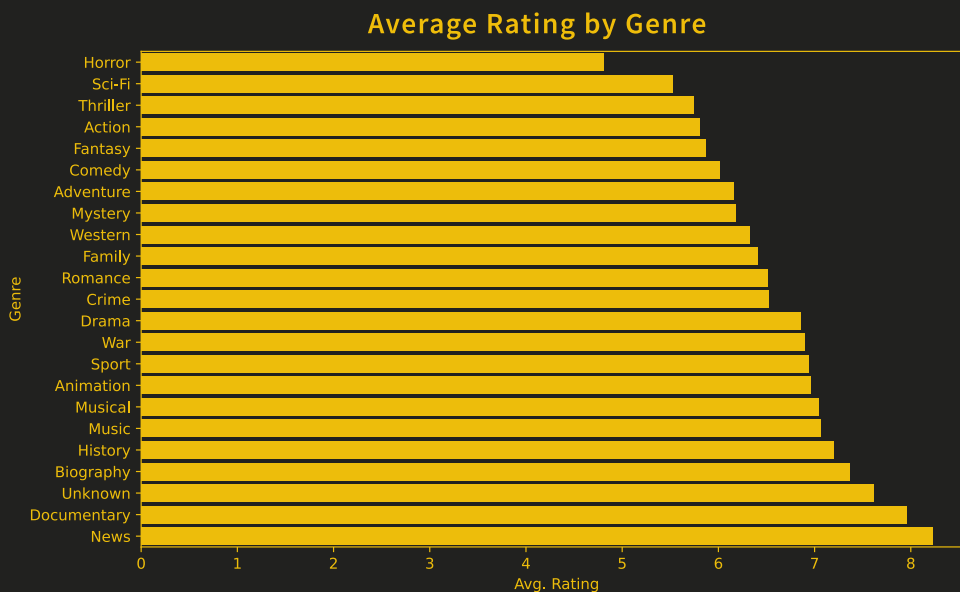
plt.figure(figsize=(10,6))
```

```
sns.barplot(x=genre_rating.values, y=genre_rating.index)

plt.title("Average Rating by Genre")
plt.xlabel("Avg. Rating")
plt.ylabel("Genre")

plt.show()
```

Insights:



- Certain genres consistently receive higher critical appreciation.
- Genres like Documentary, Biography, and History have higher average ratings.
- Genres like Horror and Sci-Fi have comparatively lower average ratings.
- Most genres fall within the 6–7.5 rating range.
- Audience preference varies significantly across genres. So, **Genre is an important feature for predicting movie ratings.**

III. Rating Trend Over Time

Why this analysis was performed?

To analyze how average movie ratings have changed over the years.

Code:

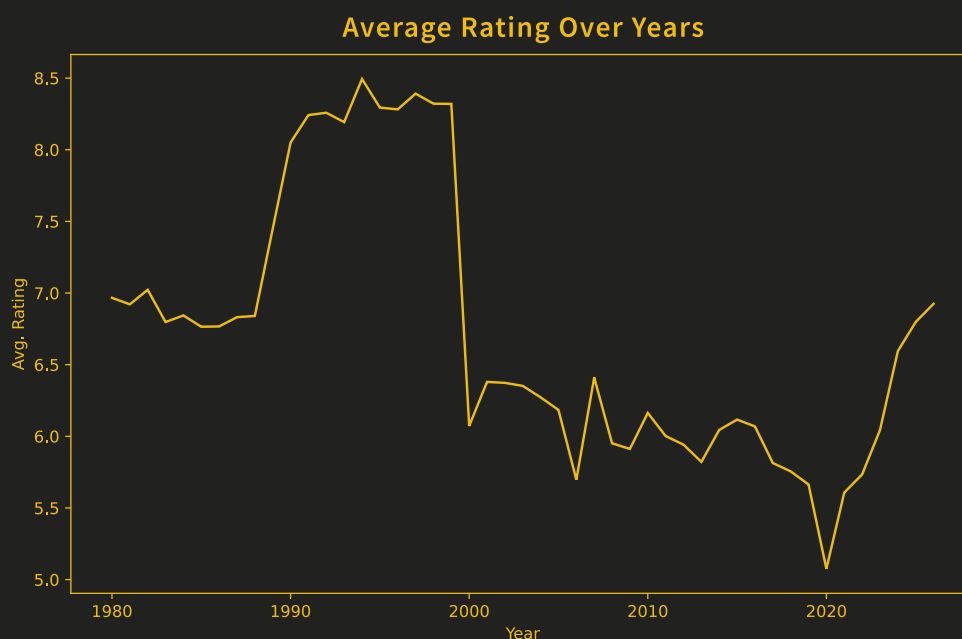
```
# Ratings Trend using Line Chart
year_rating = df.groupby('release_year')['rating'].mean()

plt.figure(figsize=(10,6))
plt.plot(year_rating)

plt.title("Average Rating Over Years")
plt.xlabel("Year")
plt.ylabel("Avg. Rating")

plt.show()
```

Insights:



- Movie ratings show temporal variation over decades.
- Ratings were relatively higher during the 1990s, while a noticeable decline occurs after the early 2000s. However, there is a slight recovery in recent years.
- Older movies tend to have higher ratings, indicates there is a possibility of selection bias.
- Time-based features can be useful for modeling trends.

IV. Budget vs Gross

Why this analysis was performed?

To understand the relationship between movie investment (budget) and revenue (gross earnings).

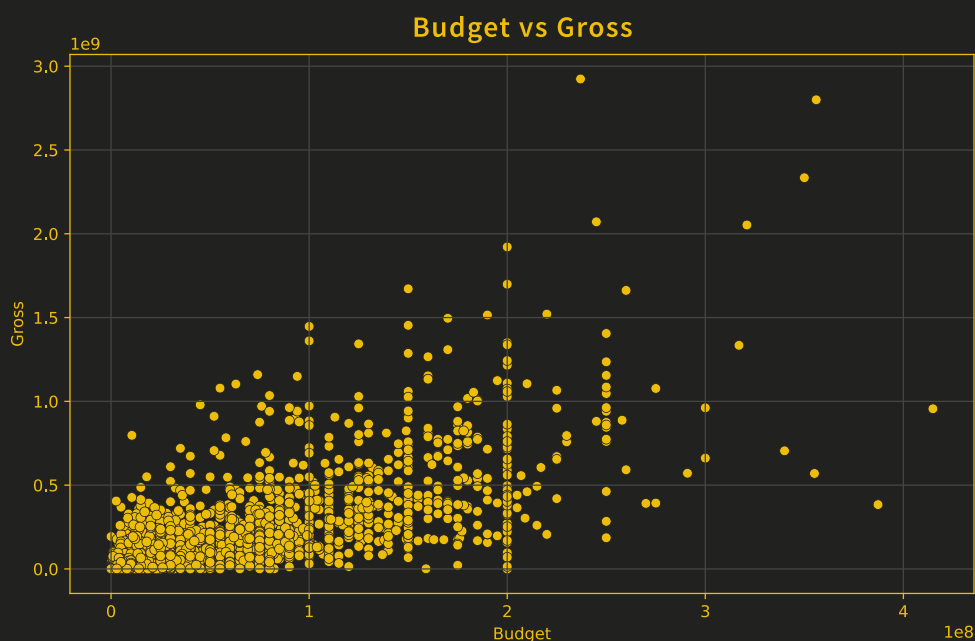
Code:

```
# Budget vs Gross using Scatterplot
plt.figure(figsize=(10,6))

sns.scatterplot(x="budget_usd", y="gross_usd", data=df)
plt.title('Budget vs Gross')
plt.xlabel('Budget')
plt.ylabel('Gross')

plt.grid()
plt.show()
```

Insights:



- There is a positive relationship between budget and gross.
- Higher budget movies tend to generate higher revenue
- Significant spread indicates variability in returns, turns out high budget does not always guarantee success.

- Presence of outliers indicates blockbuster and underperforming films.
- So, Budget is an important factor influencing gross revenue

V. Profit vs Rating

Why this analysis was performed?

To examine whether financially successful movies also receive higher ratings.

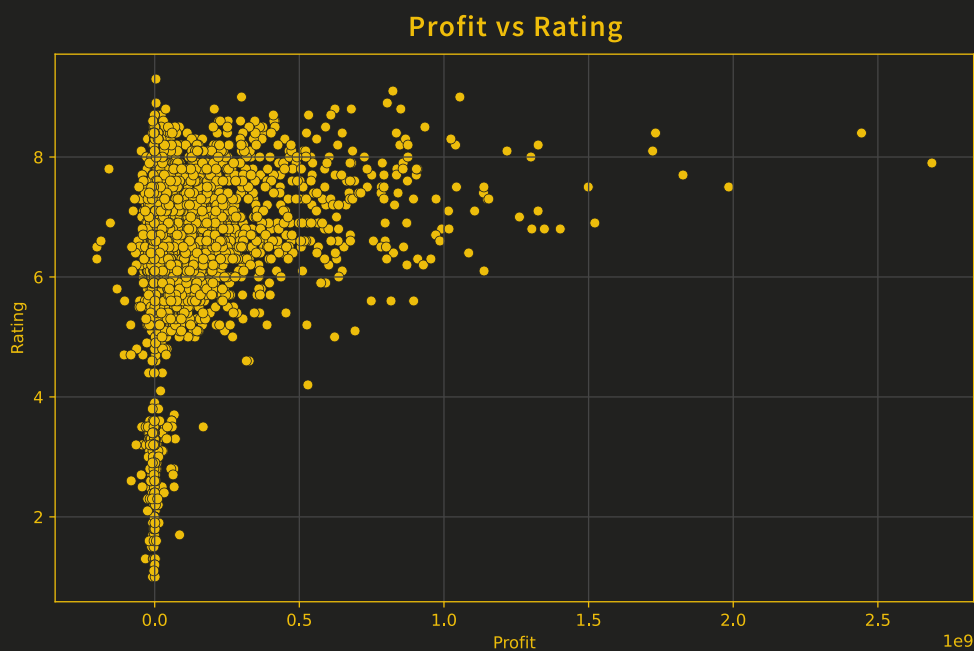
Code:

```
# Budget vs Gross using Scatterplot
df['profit'] = df['gross_usd'] - df['budget_usd']

plt.figure(figsize=(10,6))
sns.scatterplot(x="profit", y="rating", data=df)
plt.title('Profit vs Rating')
plt.xlabel('Profit')
plt.ylabel('Rating')

plt.grid()
plt.show()
```

Insights:



Insights:

- Financial success and critical success are not strongly correlated as there is no clear linear relationship between profit and rating.
 - High ratings do not necessarily imply high profitability as High-Profit movies are spread across different rating levels.
 - Some low-profit movies still have high ratings.
 - Profit alone may not be a reliable predictor of rating.
-

MULTIVARIATE ANALYSIS

Multivariate analysis examines the relationship among three or more variables to simultaneously understand complex interactions and combined effects. The objective is to:

- Identify interdependencies among multiple variables.
- Detect strong and weak relationships for feature selection.
- Understand how variables collectively influence outcomes.
- Support better decision-making for predictive modeling.

This analysis provides deeper insights compared to univariate and bivariate analysis by revealing how variables interact together.

I. Correlation Matrix Analysis

Why this analysis was performed?

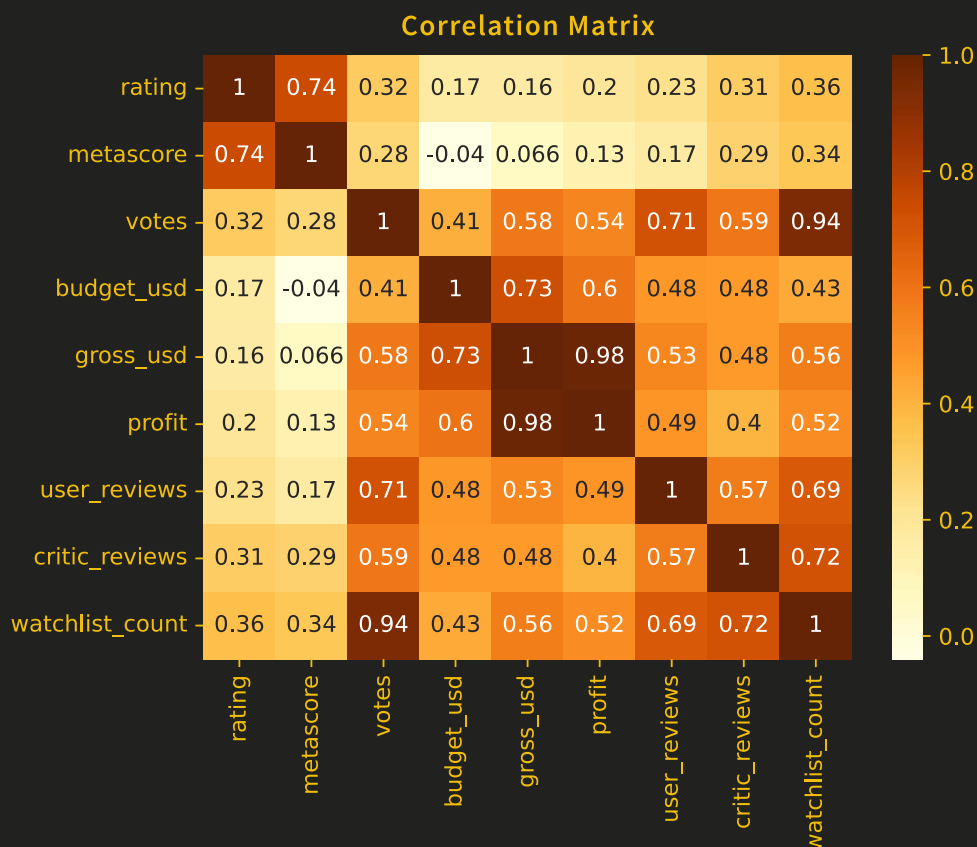
To measure the strength and direction of relationships between key numerical variables such as rating, votes, budget, gross, and profit.

Code:


```
# Correlation Heatmap
corr = df[["rating", "votes", "budget_usd", "gross_usd", "profit"]].corr()

sns.heatmap(corr, annot=True)
plt.title("Correlation Matrix")
plt.show()
```

Insights:



- **Strong positive correlation** between:
 - gross_usd & profit → **~0.98**
 - watchlist_count & votes → **~0.94**
- **Moderate correlation** between:
 - votes & gross_usd → **~0.58**
 - votes & reviews → **~0.71 & ~0.59**
- **Weak correlation** between:
 - rating with all financial variables (**~0.15-0.32**)
except metascore (~0.74)
- Budget plays a significant role in determining gross revenue.
- Ratings have weak correlation with financial metrics except metascore, indicating critically successful movies are not always financially successful.

IV. Genre vs Ratings vs Vote

Why this analysis was performed?

To analyze how genres influence both audience ratings and popularity (votes) simultaneously.

Code:

```
# Genre vs Rating vs Vote Mean
df_genre.groupby("genres").agg({
    "rating": "mean",
    "votes": "mean"
}).sort_values(by="votes", ascending=False).head(10)
```

Insights:

Genres	Rating	Votes
Adventure	6.158024	227959.444243
Sci-Fi	5.527119	227287.994350
Western	6.327273	221379.681818
Animation	6.963333	208043.270370
Fantasy	5.863927	187732.694064
Action	5.807547	180378.248935
Crime	6.524476	165163.082690
Biography	7.367622	156352.048711
War	6.892079	154812.851485
Mystery	6.183569	148553.833922

- There is a clear distinction between: **Popular genres** (higher votes) and **Critically acclaimed genres** (higher ratings).
- Example:
Action/Sci-Fi → High popularity but moderate ratings
Biography/Documentary → Lower popularity but higher ratings
- This indicates that audience engagement and critical appreciation differ across genres

CATEGORY SEGMENTATION

The objective of category-wise analysis is to examine how different predefined movies categories (such as Top Rated, Most Popular, Mid Rated, etc.) differ in terms of:

- Average ratings
- Audience engagement (votes)
- Impact of category overlap

This analysis helps in understanding how categorization influences movie performance, visibility, and audience perception.

I. Average Rating & Votes per Category

Why this analysis was performed?

To compare the quality (ratings) and audience engagement (votes) of movies across different categories.

Code:

```
# Splitting and Unpivoting Category Column
df_category = df.copy()
df_category['Scrape_Category'] = df_category['Scrape_Category'].str.split("|")
df_category = df_category.explode('Scrape_Category')

cat_rating = df_category.groupby('Scrape_Category')['rating'].mean()
cat_votes = df_category.groupby('Scrape_Category')['votes'].mean()

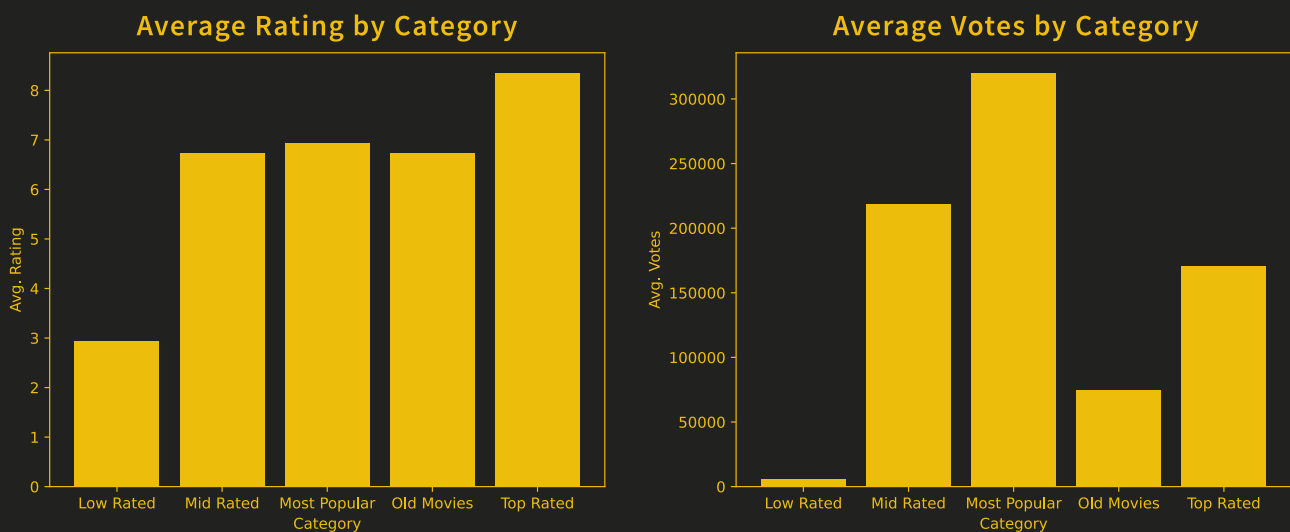
fig, (ax1,ax2) = plt.subplots(1,2, figsize=(12,5))

ax1.bar(cat_rating.index, cat_rating.values)
ax1.set_title("Average Rating by Category")
ax1.set_xlabel("Category")
ax1.set_ylabel("Avg. Rating")
```

```
ax2.bar(cat_votes.index, cat_votes.values)
ax2.set_title("Average Votes by Category")
ax2.set_xlabel("Category")
ax2.set_ylabel("Avg. Votes")

plt.tight_layout()
plt.show()
```

Insights:



- As the name suggests, Top Rated, Mid Rated, Low Rated have high, moderate, and low ratings respectively.
- Mid Rated and Popular categories overlap in terms of rating range.
- Popularity (votes) is highest for widely recognized movies (most popular movies).
- This tells high ratings do not always mean high popularity.
- Old Movies have lower engagement, likely due to limited modern audience exposure.

II. Category Count vs Rating

Why this analysis was performed?

To examine how appearing in multiple categories affects movie ratings.

Code:

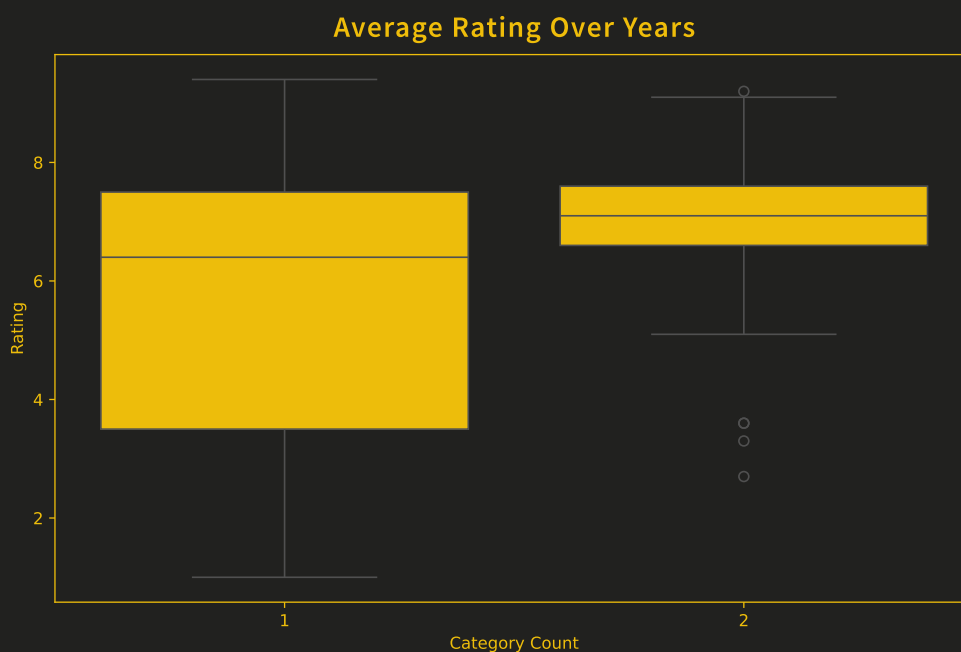
```
# Category Count Impact using Box Plot
df['category_count'] = df["Scrape_Category"].apply(lambda x: len(x.split("|")))

plt.figure(figsize=(10,6))
sns.boxplot(x="category_count", y="rating", data=df)

plt.title("Category Count vs Rating")
plt.xlabel("Category Count")
plt.ylabel("Rating")

plt.show()
```

Insights:



- Movies with category count = 2 tend to have higher median ratings.
- Movies in only one category show a wider spread, including low ratings.
- Movies appearing in multiple categories are likely:
 - More Popular
 - More Recognized
- Category overlap can be a proxy for visibility and success, indicates that multi-category presence is associated with better performance.

GENRE TREND ANALYSIS

INTRODUCTION

The objective of this analysis is to understand how audience preferences for movie genres have evolved over time. By analyzing metrics such as votes, ratings and revenue, the study aims to identify the most popular, highest quality, and most commercially successful genres, along with forecasting future trends in audience interest.

APPROACH

The Genre Trend Analysis from Data Preparation in Python to Dashboard Creation in Power BI as follows:

Data Preparation (Python):

- The raw dataset was first preprocessed to make it suitable for analysis and visualization.
 - Converted `release_date` into year
 - Split multi-value genres column into individual genres
 - Created a genre-level dataset by exploding genres
 - Columns: `movie_url`, `title`, `year`, `genres`, `rating`, `votes`, `gross_usd`
-

Data Import & Modeling:

- Imported dataset using:
 - Home → Get Data → Text/CSV
 - As, we don't have only one table, there is no need of the modeling or relationships for tables.
-

Creating Measures:

Key Measures:

Total Movies

```
Total Movies = DISTINCTCOUNT(fact_movies_genre[title])
```

Total Votes

```
Total Votes = SUM(fact_movies_genre[votes])
```

Average Rating

```
Avg Rating = AVERAGE(fact_movies_genre[rating])
```

Vote Share

```
Vote Share =  
DIVIDE(  
    SUM(fact_movies_genre[votes]),  
    CALCULATE(SUM(fact_movies_genre[votes]), ALL(fact_movies_genre[genres]))  
)
```

Top Genre

```
Top Genre =  
VAR GenreTable = SUMMARIZE(  
    fact_movies_genre,  
    fact_movies_genre[genres],  
    "TotalVotes", SUM(fact_movies_genre[votes])  
)  
VAR TopGenreRow = TOPN(1, GenreTable, [TotalVotes], DESC)  
RETURN  
    CONCATENATEX(TopGenreRow, fact_movies_genre[genres], ", ")
```

Creating Visualizations:

KPI Cards

- Total Movies
 - Total Votes
 - Average Rating
 - Top Genre
-

Top Genre by Quality (Bar Chart)

- Axis → genres
 - Values → Average Rating
-

Top Genre by Popularity (Bar Chart)

- Axis → genres
 - Values → Votes Share
-

Top Genre by Gross (Bar Chart)

- Axis → genres
 - Values → sum of gross_usd
-

Future Trend of Popularity (Line Chart)

- X-axis → year
 - Y-axis → sum of votes
 - Enable Forecasting: Confidence Interval → 95% & Seasonality → 12
-

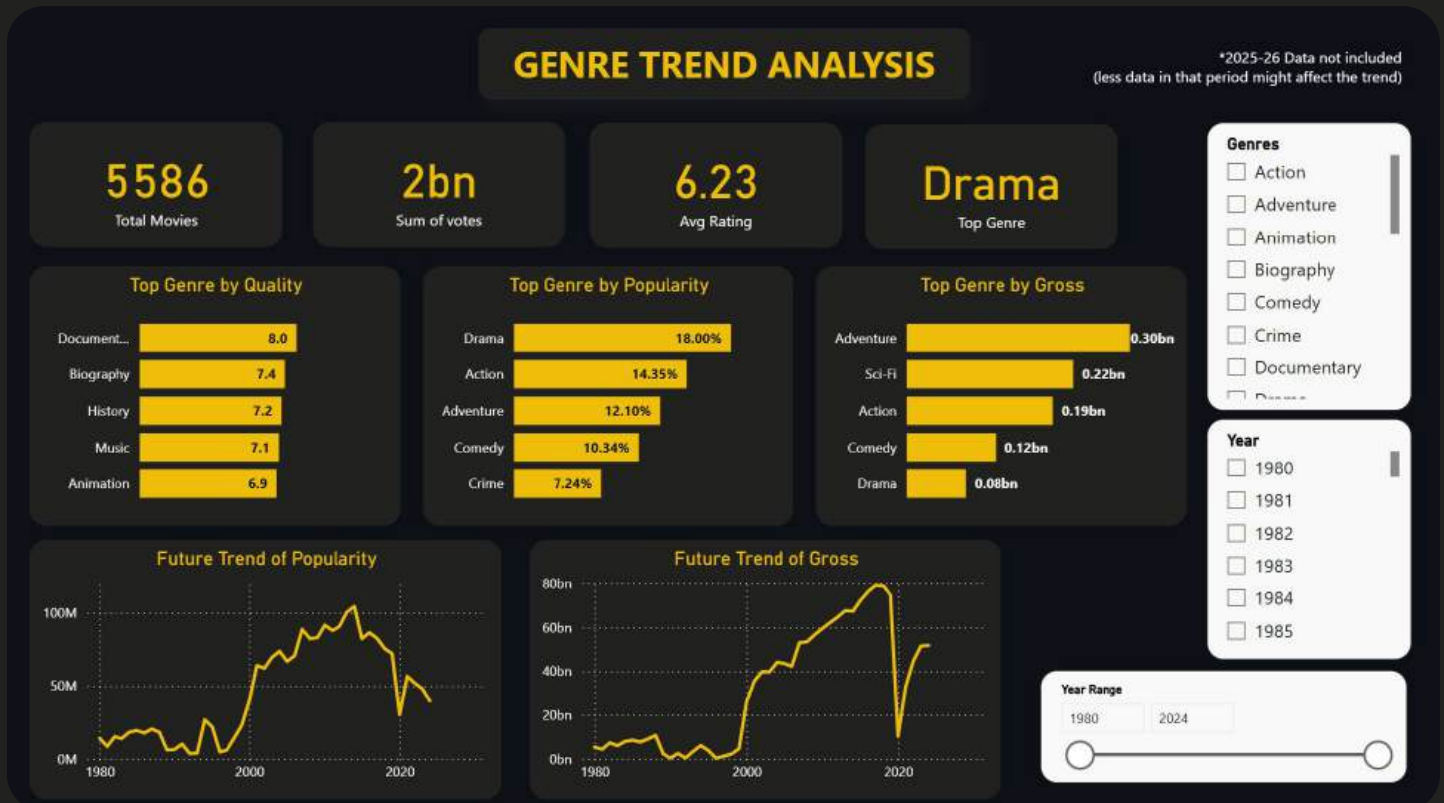
Future Trend of Popularity (Line Chart)

- X-axis → year
 - Y-axis → sum of gross_usd
 - Enable Forecasting: Confidence Interval → 95% & Seasonality → 12
-

Add Slicers

- Genre slicer
- Year slicer
- Year Range slicer

Output:



KEY INSIGHTS

Top Genre by Popularity (votes):

- Drama is the most popular genre, contributing the highest share (~18%)
- Followed by Action and Adventure
- This indicates strong audience preference for storytelling-driven and high-engagement content

Top Genre by Quality (ratings):

- Documentary ranks highest in average rating (~8.0), followed by Biography and History.
 - These genres, while less popular, are critically well-received
 - This indicates that documentary and biopic films have less audience but they love it (high average ratings).
-

Top Genre by Revenue (gross_usd):

- Adventure and Action dominate in terms of total gross revenue.
 - Comedy and Drama also contribute significantly.
 - This indicates that high-budget, large-scale genres drive box office success.
-

Genre Evolution Trends:

- Audience interest shows a steady rise from early 2000s, peaking around the late 2010s.
 - A noticeable decline after 2020, possibly due to external factors (e.g., industry shifts, streaming impact due to corona).
 - The trend indicates that genre popularity is dynamic and influenced by changing viewing patterns.
-

Popularity Forecast:

- The forecast shows uncertainty and fluctuation, indicating unstable future audience interest.
 - Suggests that no single genre is guaranteed dominance going forward.
-

Revenue Forecast:

- Revenue trends show high volatility, with sharp fluctuations in recent years.
- This indicates dependency on blockbuster releases and external market factors.

RATING PREDICTION

INTRODUCTION

The modelling phase focuses on building predictive models to extract meaningful insights and make data-driven predictions from the IMDb dataset. After completing data cleaning, exploratory analysis, and visualization, machine learning techniques were applied to understand the relationship between movie attributes and their ratings.

This phase includes:

- Feature selection and engineering
- Handling skewness and outliers
- Encoding categorical variables
- Model training and optimization
- Performance evaluation on both test and unseen data

The primary goal is to develop a model that can accurately predict IMDb ratings and generalize well to new data.

APPROACH

A structured machine learning pipeline was followed:

Feature Engineering:

- Dropped the rows having null values in budget & gross column.
- Log transformation applied to skewed features like votes, gross, and budget to reduce the skewness.
- Extreme values in numerical columns were capped

Categorical Encoding:

- Genres were encoded using multi-label binarization to handle multiple genres per movie.
 - Content rating was one-hot encoded.
 - Director and lead actor were encoded using mean rating (target encoding).
-

Feature Selection:

- Redundant and non-informative columns were removed.
 - Only model-relevant numerical and encoded features were retained.
-

Model Selection:

- Linear Regression was used as a baseline model.
 - Random Forest Regressor was used to capture non-linear relationships.
-

Hyper Parameter Tuning:

- GridSearchCV was applied to optimize Random Forest parameters.
-

Model Evaluation:

- Model evaluation using:
 - MSE
 - RMSE
 - MAE
 - R^2 score.
- External validation on unseen 2025 movie dataset.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# importing required libraries
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import GridSearchCV
from sklearn.preprocessing import MultiLabelBinarizer
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# load cleaned csv file
df = pd.read_csv('data/IMDb_movies_cleaned3.csv')
df.head()

# Convert to datetime & extract year
df["release_date"] = pd.to_datetime(df["release_date"], errors="coerce")
df['release_year'] = df['release_date'].dt.year

# Drop records with null values in gross and budget
df = df.dropna(subset=['gross_usd', 'budget_usd'])

#----- FEATURE ENGINEERING -----

# Log Transform to handle skewness
df['log_votes'] = np.log1p(df['votes'])
df['log_gross'] = np.log1p(df['gross_usd'])
df['log_budget'] = np.log1p(df['budget_usd'])
df['log_user_reviews'] = np.log1p(df['user_reviews'])
df['log_critic_reviews'] = np.log1p(df['critic_reviews'])
df['log_watchlist_count'] = np.log1p(df['watchlist_count'])

# capping votes
df['log_votes'] = df['log_votes'].clip(upper=df['log_votes'].quantile(0.95))

# Genre Encoding using MultiLabelBinarizer
df['genres'] = df['genres'].str.split('|')
```

```

mlb = MultiLabelBinarizer()
genre_encoded = pd.DataFrame(
    mlb.fit_transform(df['genres']),
    columns=mlb.classes_,
    index=df.index
)
df = pd.concat([df, genre_encoded], axis=1)

# Content Rating Encoding using One Hot Encoding
df = pd.get_dummies(df, columns=['content_rating'], drop_first=False)

# Director Encoding (Mean score)
director_mean = df.groupby('director')['rating'].mean()
df['director_score'] = df['director'].map(director_mean)

# Star Encoding (Mean score)
star_mean = df.groupby('star_1')['rating'].mean()
df['star_score'] = df['star_1'].map(star_mean)

#----- FEATURE SELECTION -----

drop_cols = [
    'title', 'genres', 'votes', 'movie_url', 'currency_type', 'release_date',
    'gross', 'budget', 'gross_usd', 'budget_usd', 'Scrape_Category', 'keywords',
    'director', 'description', 'countries', 'languages', 'watchlist_count',
    'star_1', 'star_2', 'star_3', 'user_reviews', 'critic_reviews'
]
df_model = df.drop(columns=drop_cols, errors='ignore')

#----- OUTLIER TREATMENT -----

num_cols = ['log_votes', 'log_gross', 'log_budget', 'duration', 'metascore',
            'log_critic_reviews', 'log_user_reviews', 'log_watchlist_count']
for i in num_cols:
    sns.boxplot(x=df_model[i])
    plt.show()

def outliers_treatment(x):
    x=x.clip(upper=x.quantile(0.98))
    x=x.clip(lower=x.quantile(0.02))
    return x

df_model[num_cols] = df_model[num_cols].apply(outliers_treatment)

```

```

# Correlation Heatmap
sns.heatmap(df_model[['rating','duration','log_votes', 'log_gross', 'log_budget',
'release_year']].corr(), annot=True)
plt.show()

#----- TRAIN TEST SPLIT & SCALING -----

X = df_model.drop(columns=['rating'])
y = df_model['rating']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

#----- MODEL TRAINING -----

# Linear Regression
lr = LinearRegression()
lr.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)
y_pred_lr_train = lr.predict(X_train)

#----- HYPER PARAMETER TUNING -----

param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [8, 10, 15],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2],
    'max_features': ['sqrt', 'log2']
}

# Random Forest Regressor
rf = RandomForestRegressor(random_state=42)

```

```

grid_search = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    cv=3,
    scoring='r2',
    n_jobs=-1,
    verbose=2
)

grid_search.fit(X_train_scaled, y_train)

print("Best Parameters:")
print(grid_search.best_params_)
best_rf = grid_search.best_estimator_

y_pred_rf = best_rf.predict(X_test)
y_pred_rf_train = best_rf.predict(X_train)

#----- HYPER PARAMETER TUNING -----

def evaluate_model(name, y_true, y_pred):
    print(f"\n{name} Performance:")
    print("MSE:", round(mean_squared_error(y_true, y_pred),3))
    print("RMSE:", round(np.sqrt(mean_squared_error(y_true, y_pred)),3))
    print("MAE:", round(mean_absolute_error(y_true, y_pred), 3))
    print("R2 Score:", round(r2_score(y_true, y_pred), 3))

print("-----Training Data-----")
evaluate_model("Linear Regression", y_train, y_pred_lr_train)
evaluate_model("Random Forest", y_train, y_pred_rf_train)

print("\n-----Testing Data-----")
evaluate_model("Linear Regression", y_test, y_pred_lr)
evaluate_model("Random Forest", y_test, y_pred_rf)

#----- HYPER PARAMETER TUNING -----

importance = pd.Series(best_rf.feature_importances_, index=X.columns)
importance = importance.sort_values(ascending=False)

print("\nTop 10 Important Features:")
print(importance.head(10))

```


Output:

Output:

-----Training Data-----

Linear Regression Performance:

MSE: 0.132

RMSE: 0.364

MAE: 0.27

R2 Score: 0.917

Random Forest Performance:

MSE: 0.033

RMSE: 0.181

MAE: 0.134

R2 Score: 0.98

-----Testing Data-----

Linear Regression Performance:

MSE: 0.132

RMSE: 0.363

MAE: 0.277

R2 Score: 0.921

Random Forest Performance:

MSE: 0.131

RMSE: 0.362

MAE: 0.27

R2 Score: 0.921

-----Feature Importance-----

Top 10 Important Features:

Output:

```
director_score      0.271637
star_score          0.204419
log_watchlist_count 0.110549
metascore           0.101408
log_votes           0.075982
log_critic_reviews  0.043217
log_user_reviews    0.039144
duration            0.031131
log_gross           0.022043
log_budget          0.019394
dtype: float64
```

EVALUATION RESULTS & INSIGHTS

The results of both models are as follows:

Model Performance:

- Random Forest outperformed Linear Regression due to its ability to capture complex, non-linear relationships.
- High R^2 scores ($\sim 0.90+$) were achieved on training and test datasets, indicating strong predictive capability.
- Linear Regression showed stable but slightly lower performance.

Key Feature Insights:

- Director & Cast emerged as one of the strongest predictors.
- Review features like watchlist count and metascore also contributes well.
- Some Reviews features (User_Reviews, Crittic_Reviews) contributed moderately.
- Gross, Budget, and Duration had minimal impact on ratings.

Test on New Unseen Data:

- Both Linear Regression and Random Forest models were tested on new 2025-2026 movies data which is not in dataset.
 - Both the models gave very low R2 score (~ 0.3) and high error.
 - This suggests that both the **models were overfitted**.
 - As the movies rating depend on many external factors, and also our dataset is biased, and less no. of movies data, it memorized all the data.
 - Even advance models cannot perfectly predict the movie ratings without the proper user behavioral data.
-

RECOMMENDATION SYSTEM

INTRODUCTION

The objective of this phase is to build a content-based movie recommendation system that suggests similar movies based on:

- Genre & keywords
- Description (plot)
- Cast & director
- Engagement metrics (rating, votes, watchlist, reviews)
- Country & language

APPROACH

We will use a Hybrid Content-Based Filtering approach:

- Text similarity → captures story/theme
- Categorical features → captures structure (genre, country)
- Numeric features → captures popularity

At last we'll find the similarity based on the combination of all these signals.

Feature Engineering:

Text Features

- description
- keywords
- Useful to capture story or theme.

Categorical Features

- genres
- content_rating
- countries
- language
- Useful to capture the structure of movies.

Cast & Crew

- director
- star_1
- Useful to capture the similar director or movies

Numeric Features

- rating
- votes
- watchlist count
- user reviews count
- critic reviews count
- metascore
- Useful to capture popularity & quality.

Data Preprocessing:

Data preprocessing ensures uniformity and consistency, which is essential for accurate feature transformation and similarity calculation. This includes:

- cleaned text columns (replaced Unknown value)
- Converted genres, countries, languages to list format
- Handled missing values in metascore column with the average of the median values of genre's metascore.
- Eg., 'Action|Adventure|Drama' → avg. of the median values of Action, Adventure and Drama Genre

Code:

```
import pandas as pd
import numpy as np

# load cleaned csv file
df = pd.read_csv('data/IMDb_movies_cleaned.csv')

# Fill null values in keywords with empty string
df['keywords'] = df['keywords'].fillna('')

# Drop records where genre is Unknown
df = df[df['genres'] != 'Unknown']

#----- Handling Missing Metascore -----

# Explode genres to calculate median value of each genre
df_genre = df.copy()
df_genre['genres'] = df['genres'].str.split("|")
df_genre = df_genre.explode('genres')

genre_metascore = df_genre.groupby('genres')['metascore'].median().astype(int)

# Calculate from the string (e.g., 'Action|Adventure')
def calculate_avg_metascore(x):
    genres_list = str(x).split('|')
    scores = []

    for g in genres_list:
        if g in genre_metascore:
            scores.append(genre_metascore[g])

    # If no valid genre impute global median
    if len(scores) == 0:
        return df['metascore'].median()

    return sum(scores) / len(scores)

# Imputing to null values
df['metascore_imputed'] = df['metascore'].copy()
mask = df['metascore'].isna()
```

```

df.loc[mask, 'metascore_imputed'] = df.loc[mask, 'genres'].apply(
    calculate_avg_metascore)

df['metascore'] = df['metascore_imputed']
df.drop(columns=['metascore_imputed'], inplace=True)

# Replace Unknown values with empty string
for col in ['director', 'star_1', 'star_2',
            'star_3', 'content_rating', 'languages']:
    df[col] = df[col].str.replace('Unknown', '')

# convert categorical features to list
df['genres'] =
df['genres'].apply(lambda x: x.split('|') if isinstance(x, str) else [])

df['countries'] = df['countries'].apply(lambda x: x.split('|'))
df['languages'] = df['languages'].apply(lambda x: x.split('|'))

```

Text Vectorization:

TF-IDF converts text into numerical form while giving higher importance to unique words and reducing the effect of common words, improving similarity detection between movies.

- Before applying TF-IDF, first combine all the features to make a text from these columns:
 - description, keywords, genres, director, stars, content rating

Code:

```

df['combined_text'] = (
    df['description'] + " " +
    df['keywords'] + " " +
    df['genres'].apply(lambda x: " ".join(x)) + " " +
    df['director']*2 + " " +
    df['star_1']*2 + " " +
    df['star_2'] + " " +
    df['star_3'] + " " +
    df['content_rating']
)

```

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(
    stop_words='english',
    max_features=5000,
    ngram_range=(1,2)
)

tfidf_matrix = tfidf.fit_transform(df['combined_text'])
```

MultiLabel Encoding:

Movies can belong to multiple genres. MultiLabel encoding allows us to represent multiple categories without losing information, unlike Label Encoding. Used it to encode:

- genres
- countries
- languages

Code:

```
from sklearn.preprocessing import MultiLabelBinarizer

mlb = MultiLabelBinarizer()

genre_matrix = mlb.fit_transform(df['genres'])
country_matrix = mlb.fit_transform(df['countries'])
language_matrix = mlb.fit_transform(df['languages'])
```

Numerical Features:

Multiple numerical features like rating, metascore, votes, watchlist_count, user_reviews, critic_reviews to capture the popularity and quality for the recommended movies.

Standard Scaler is used to encode these numerical features.


```
from sklearn.preprocessing import StandardScaler

num_cols = [
    'rating', 'votes', 'watchlist_count',
    'user_reviews', 'critic_reviews', 'metascore'
]

scaler = StandardScaler()
num_matrix = scaler.fit_transform(df[num_cols])
```

Filtering:

To improve recommendation relevance, a hybrid approach was implemented by combining content similarity with genre-based filtering, and language-based filtering ensuring that recommendations remain contextually and categorically aligned.

Code:

```
def filter_by_genre(movie_idx):

    movie_genres = set(df.loc[movie_idx, 'genres'])

    return df[df['genres'].apply(
        lambda x: len(set(x) & movie_genres) >= 2
    )].index

def filter_by_language(movie_idx, indices):

    movie_lang = df.loc[movie_idx, 'languages']

    return [i for i in indices if movie_lang in df.loc[i, 'languages']]
```

Feature Combination & Weighting:

A unified representation is required to compute similarity between movies across multiple dimensions (text + categorical + numeric).

- Combined all features into a single feature matrix
- Without weighting, numeric features (like votes, rating) dominate the model. Weighting ensures content relevance is prioritized over popularity bias.

Code:

```
from scipy.sparse import hstack

final_matrix = hstack([
    tfidf_matrix * 0.6,
    genre_matrix * 0.3,
    country_matrix * 0.02,
    language_matrix * 0.03,
    num_matrix * 0.05
])
```

Similarity Calculation:

Cosine similarity measures the angle between feature vectors, making it ideal for high-dimensional sparse data like TF-IDF, and focuses on similarity rather than magnitude.

Code:

```
from sklearn.metrics.pairwise import cosine_similarity

similarity = cosine_similarity(final_matrix)
```

Recommendation Generation:

Movies with highest similarity scores are considered most relevant, ensuring accurate and meaningful recommendations.

Selected top N most similar movies.

Code:

```
def recommend(movie_name, top_n=5):

    if movie_name not in df['title'].values:
        return f"Movie '{movie_name}' not found in dataset"
    idx = df[df['title'] == movie_name].index[0]

    filtered_indices = filter_by_genre(idx)
    filtered_indices = filter_by_language(idx, filtered_indices)

    sim_scores = list(enumerate(similarity[idx]))
    sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)[1:top_n+1]

    movie_indices = [i[0] for i in sim_scores]
    result = df.iloc[movie_indices][['title', 'genres', 'director', 'rating',
    'metascore', 'movie_url']]

    return result
```

Output:

Recommend("Spider-Man", Top_n=10)

title	genres	rating	metascore
Spider-Man 2	[Action, Adventure, Sci-Fi]	7.5	83
Spider-Man 3	[Action, Adventure, Sci-Fi]	6.3	59
The Amazing Spider-Man	[Action, Sci-Fi]	6.9	66
The Amazing Spider-Man 2	[Action, Adventure, Sci-Fi]	6.6	53
X-Men	[Action, Adventure, Sci-Fi]	7.3	64
Captain Marvel	[Action, Adventure, Sci-Fi]	6.7	64
Venom	[Action, Adventure, Sci-Fi]	6.6	35
Spider-Man: Homecoming	[Action, Adventure, Sci-Fi]	7.4	73
Spider-Man: Far From Home	[Action, Adventure, Sci-Fi]	6.4	44
Man of Steel	[Action, Adventure, Sci-Fi]	7.1	55

Recommend(“Toy Story”, Top_n=10)

title	genres	rating	metascore
Toy Story 2	[Animation, Adventure, Comedy]	7.9	88
Toy Story 3	[Animation, Adventure, Comedy]	8.3	92
Toy Story 4	[Animation, Adventure, Comedy]	7.6	84
Finding Nemo	[Animation, Adventure, Comedy]	8.2	90
Up	[Animation, Adventure, Comedy]	8.3	88
Monster Inc.	[Animation, Adventure, Comedy]	8.1	79
Inside Out	[Animation, Adventure, Comedy]	8.1	94
Ratatouille	[Animation, Adventure, Comedy]	8.1	96
Cars	[Animation, Adventure, Comedy]	7.3	73
Shrek	[Animation, Adventure, Comedy]	7.9	84

DASHBOARD CREATION

INTRODUCTION

The objective of this phase is to design an interactive Power BI dashboard that present key insights, trends, recommendations derived from the IMDb dataset. The dashboard aims to:

- Provide a comprehensive overview of movie performance metrics such as ratings, votes, and revenue.
- Visualize genre trends and patterns across different time periods.
- Showcase a recommendation system for suggesting similar movies.
- Enable users to explore data dynamically using interactive filters and advanced reporting features

RLS was not able to performed, as we used Power BI Desktop not Power BI Service.

APPROACH

Data Preparation & Integration:

The cleaned and processed datasets from Python were exported into multiple structured tables, including:

- Main movie dataset (features and metrics)
- Genre-level dataset (for trend analysis)
- Genre-level dataset (for trend analysis)

These tables were imported into Power BI and appropriate relationships were established (e.g., using movie_url as a unique identifier) to ensure accurate data mapping across visuals.

Data Modeling:

To support advanced reporting:

- A normalized genre table was created to handle multi-valued genre fields and enable effective filtering.
 - Relationships were designed carefully to manage one-to-many mappings between movies and recommendations.
 - Additional calculated columns (e.g., recommended movie attributes) were created using DAX functions like **LOOKUPVALUE**.
-

Visualization Techniques:

Various visualizations were used to represent data effectively:

- KPI cards for summary metrics
 - Line and bar charts for trend analysis
 - Scatter plots for relationship analysis
 - Scatter plots for relationship analysis
 - AI visual (Key Influencer) for relationships
-

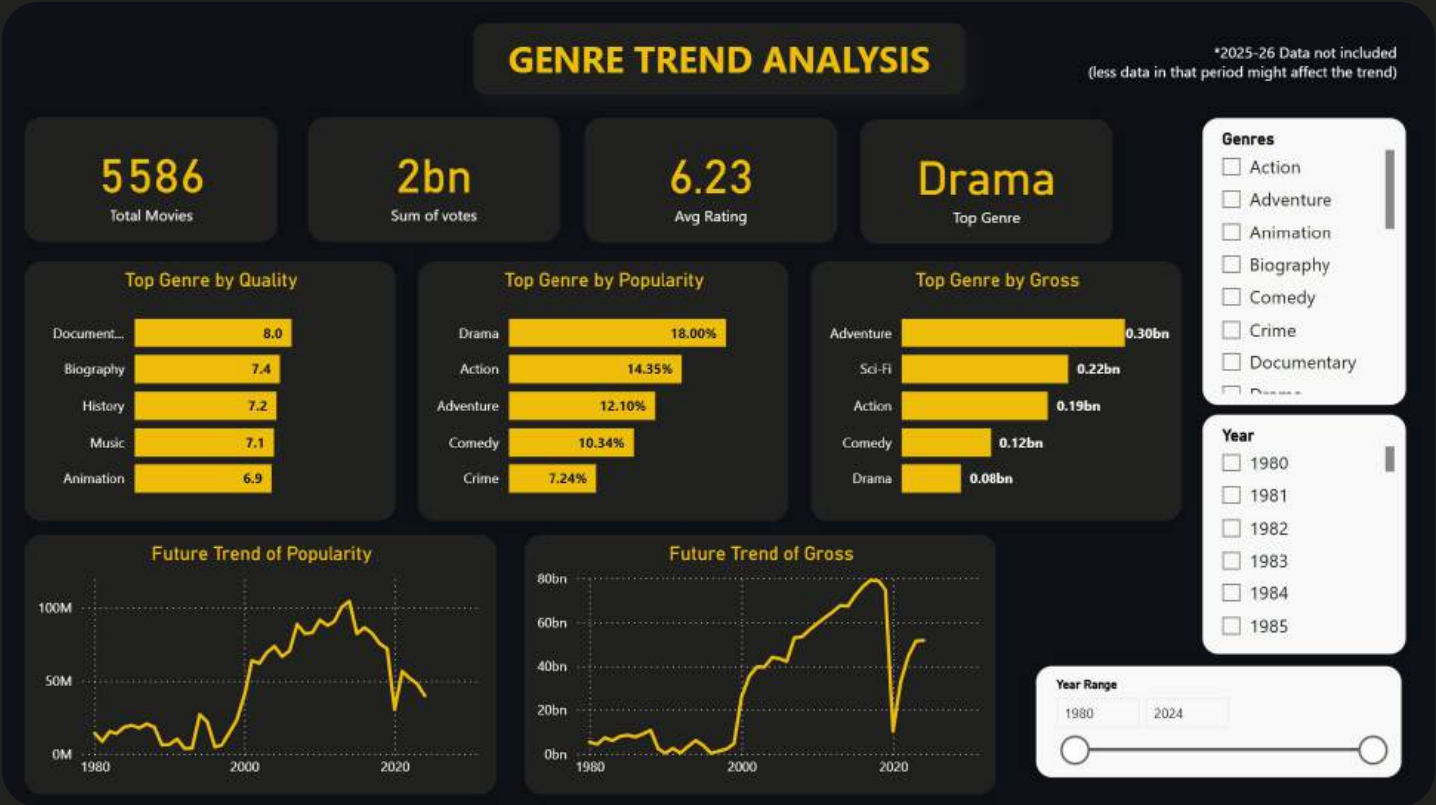
Interactive Features:

To enhance user experience and usability, several Power BI features were implemented:

- Slicers (Genre, Year) for dynamic filtering
 - Sync Slicers to maintain consistent filters across pages
 - Bookmarks to toggle between different analytical views (e.g., Rating vs Votes, Budget, Gross)
 - Drill-through functionality to view detailed recommendations for selected movies
-

Output:

Genre Trend Analysis



Movies List

MOVIES LIST

Highest Rated Movies	
title	Rating
Golden Opulence: 500 Years of Luxury in Anatolia	9.40
Janam	9.30
The Shawshank Redemption	9.30
Shingeki no Kyojin: The Last Attack	9.20
Akkada Varu Ikkada Unnaru	9.10
Chitti - Potti	9.10
National Theatre Live: Prima Facie	9.10
Ramayana: The Legend of Prince Rama	9.10
The Dark Knight	9.10
Aguner Poroshmoni	9.00
Ashi Hi Banwa Banwi	9.00
Bhale Unnade	9.00
Paagal vs Kadhal	9.00
Paint Drying	9.00
Sandesham	9.00
Schindler's List	9.00
The Lord of the Rings: The Return of the King	9.00
THEVR10: A dokumentumfilm	9.00
Usha Parinayam	9.00
Vreme na nasilie	9.00

Most Popular Movies	
title	votes
The Shawshank Redemption	3.17M
The Dark Knight	3.15M
Inception	2.80M
Fight Club	2.59M
Interstellar	2.50M
Forrest Gump	2.48M
Pulp Fiction	2.42M
The Matrix	2.24M
The Lord of the Rings: The Fellowship of the Ring	2.20M
The Lord of the Rings: The Return of the King	2.15M
Se7en	2.01M
The Dark Knight Rises	1.98M
The Lord of the Rings: The Two Towers	1.95M
Django Unchained	1.87M
Gladiator	1.84M
The Wolf of Wall Street	1.78M
Inglourious Basterds	1.77M
The Silence of the Lambs	1.71M
Batman Begins	1.71M
Joker	1.68M

Highest Grossing Movies	
title	gross_usd
Avatar	\$2.92bn
Avengers: Endgame	\$2.80bn
Avatar: The Way of Water	\$2.33bn
Titanic	\$2.26bn
Star Wars: Episode VII - The Force Awakens	\$2.07bn
Avengers: Infinity War	\$2.05bn
Spider-Man: No Way Home	\$1.92bn
Inside Out 2	\$1.70bn
Jurassic World	\$1.67bn
The Lion King	\$1.66bn
The Avengers	\$1.52bn
Fast & Furious 7	\$1.52bn
Top Gun: Maverick	\$1.50bn
Frozen II	\$1.45bn
Barbie	\$1.45bn
Avengers: Age of Ultron	\$1.41bn
The Super Mario Bros. Movie	\$1.36bn
Black Panther	\$1.35bn
Harry Potter and the Deathly Hallows: Part 2	\$1.34bn
Deadpool & Wolverine	\$1.34bn

Recommendations

RECOMMENDATION

MOVIE DETAILS							
title	genres	director	star_1	rating	votes	metascore	gross_usd
Jurassic World	Action Adventure Sci-Fi	Colin Trevorrow	Chris Pratt	6.90	0.73M	59	\$1.67bn

RECOMMENDED MOVIES							
recommended_movie	genres	director	star_1	rating	votes	metascore	gross_usd
Star Trek	Action Adventure Sci-Fi	J.J. Abrams	Chris Pine	8	0.64M	82	\$0.39bn
Jurassic World: Fallen Kingdom	Action Adventure Sci-Fi	J.A. Bayona	Chris Pratt	6	0.38M	51	\$1.31bn
Jurassic World: Dominion	Action Adventure Sci-Fi	Colin Trevorrow	Chris Pratt	6	0.24M	38	\$1.00bn
Jurassic Park III	Action Adventure Sci-Fi	Joe Johnston	Sam Neill	6	0.37M	42	\$0.37bn
Jurassic Park	Action Adventure Sci-Fi	Steven Spielberg	Sam Neill	8	1.17M	68	\$1.10bn

VIDEO PRESENTATION

INTRODUCTION

The objective of this task is to evaluate the project or give the presentation through the video explaining, giving brief idea about all the key findings or insights from the analysis you have done.

DRIVE LINK

<https://drive.google.com/file/d/1IAVanXlwS34PDMmqaZoby2vpN0M3rp6y/view?usp=sharing>